

И еще немного извращений из мира прокси и VPN / Хабр

Жизнь диссидента-экспериментатора не сидит на месте, а копилка безумных находок пополняется быстрее, чем успеваешь про них написать. Продолжаем то, на чем остановились в прошлый раз - туннели через ICMP, DNS и serverless-функции это, конечно, хорошо, но вот вам еще три свежие находки: туннель через что угодно, что умеет хранить файлы, туннель через почтовый ящик, и реанимация древней технологии обхода через CDN и не только, которую почему-то все забыли.

File Tunnel

Идея максимально простая и от того гениальная: если два хоста имеют доступ к одному и тому же файловому хранилищу, то можно использовать обычные файлы для установления сетевой связности между этими хостами. File-Tunnel слушает локальный TCP-порт, и когда на него приходит соединение, все передаваемые данные просто дописываются в файл. Тот же самый файл читает другая копия File-Tunnel на другой стороне, восстанавливает из него TCP-поток и прокидывает его дальше, куда скажут. В обратную сторону данные летят точно так же, только через второй файл. И чтобы файлы не росли бесконечно, он периодически подчищаются/ротятся.

“Файловое хранилище, до которого дотягиваются оба хоста” может быть чем угодно: расшаренная папка по SMB или NFS, FTP-сервер, диск, проброшенный внутрь RDP или Citrix-сессии, VirtualBox shared folder, WSL, Docker volume, Dropbox.

Но самое интересное случилось совсем недавно: в проект добавили нативную поддержку S3 и WebDAV. И работает все это довольно неплохо.

И вот тут кроется самое важное: S3-совместимое хранилище сегодня предлагает вообще довольно много кто, и обычно за очень вменяемые деньги (а часто и дают что-то типа 1 гигабайта в рамках бесплатного тарифа) - достаточно завести бакет у любого более-менее приличного облачного провайдера, и у вас уже есть транспорт для туннеля, который снаружи выглядит как самые обычные обращения к объектному хранилищу.

Запускается это все примерно так:

Сторона А:

```
ft.exe --s3 --bucket mybucket --region ru-moscow-1 --  
access-key XXXXXXXX --secret-key XXXXXXXX -L  
5000:192.168.1.20:3389
```

Сторона В:

```
ft.exe --s3 --bucket mybucket --region ru-moscow-1 --  
access-key XXXXXXXX --secret-key XXXXXXXX
```

И для не-AWS хранилищ нужно добавить --endpoint. Ключи можно (и нужно) не светить в командной строке, а вынести в переменные окружения FT_S3_ACCESS_KEY / FT_S3_SECRET_KEY.

С WebDAV все еще проще - подойдет даже... Mail.ru Cloud:

```
ft.exe --webdav --url https://yoururl.com/path -u foo -p bar -L 5000:192.168.1.20:3389
```

True IMAP Tunnel (Secure)

Помните Bridge To Freedom из прошлой статьи - туннель через вебсокеты в Яндекс Serverless Functions? У его автора вышел еще один снаряд в ту же степь, только на этот раз в качестве транспорта используется... электронная почта. Встречайте: True IMAP Tunnel (Secure).

Идея в лоб: оба конца туннеля логинятся в один и тот же IMAP-аккаунт на почтовом сервере (или на нескольких для распараллеливания), и используют по две выделенные папки на каждый - в одну сторону и в другую. Пакет данных превращается в маленький бинарный фрейм, фрейм заворачивается в черновик письма и кладется в папку через APPEND. Ни одно из этих "писем" никогда не уходит по SMTP и никому не доставляется - это просто хранилище байт внутри почтового ящика, доступное с обеих сторон. Получатель через IDLE (либо через обычный polling, если сервер не поддерживает IDLE) видит новый черновик, вытаскивает из него фрейм, достает из него данные, помечает письмо как удаленное и позже устраивает EXPUNGE пачкой. В общем, обычный TCP по своей сути, только вместо сокета - папка на почтовом сервере.

Опционально все фреймы можно шифровать AES-256-GCM перед тем как класть в ящик (пустой passphrase выключает шифрование), а еще можно настроить, как именно "письмо" будет выглядеть для того, кто вдруг откроет эту папку в обычном почтовом клиенте - тему, вложение вместо текста, имя файла и так далее, чтобы туннель не так сильно бросался в глаза.

Автор тестировал все это дело с разными провайдерами: Gmail, Outlook, Seznam, Mail.ru, Yandex, Timeweb, Rambler -

какие-то работают быстро и достаточно стабильно, какие-то не очень.

Программа - это один и тот же Go-бинарник для клиента и сервера, туннелирующий произвольный TCP: SOCKS5 (тот же Dante), VLESS (Xray/sing-box), SSH - что угодно.

А еще это все дело можно запустить как плагин Shadowsocks (SIP003), например под Shadowsocks Android, так что пользоваться можно и с телефона.

В README автор отдельно подмигивает: аббревиатура "True IMAP Tunnel (Secure)" была выбрана не случайно.

Meek

А теперь об олдскульных идеях. ХНТТР - это очень хорошо, он довольно эффективен и пролезает через многие CDN. А через многие не пролезает (например, если CDN не любит очень долго висящие HTTP-сессии с передачей данных только в одну сторону, или если она буферизует передаваемые данные). Да и CDN в мире и в стране существует не то чтобы прям много разных. И тут самое время вспомнить про то, чем еще деды пробивали блокировки: Meek из проекта Tor.

В сравнении с ХНТТР, там все довольно примитивно устроено на уровне протокола: клиент шлет обычные HTTP POST-запросы (тело до 64 КБ) на URL, добавляя заголовок X-Session-Id, чтобы сервер понимал, какому TCP-соединению эти байты принадлежат. Сервер держит в памяти таблицу "X-Session-Id -> открытое TCP-соединение", пишет туда пришедшие байты, недолго ждет, не появилось ли что-то в ответ, и отдает это в теле HTTP-ответа. Если клиенту нечего передать, он все равно продолжает слать пустые POST-запросы, потому что HTTP - протокол с инициативой на стороне клиента; интервал polling-а плавно растет с 100 мс до 5 секунд на простое.

И благодаря этой простоте, с помощью Meek можно проксироваться (хоть и не очень быстро и существенными задержками) даже не только через CDN, но и через тупые shared-хостинги - достаточно чтобы там была возможно запустить небольшой PHP или Python скрипт (в meek они называются reflectors и их можно найти в репе meek).

Meek предполагается использовать как pluggable transport для Tor, но на самом деле можно его обмануть и заставить работать и без Tor. meek-client и meek-server ожидают на входе набор переменных окружения формата TOR_PT_*, которые Tor обычно сам подсовывает дочернему процессу управляемого транспорта. Если замочать эти переменные руками, получаются два совершенно самостоятельных бинарника, которые просто говорят друг с другом по HTTPS, и никакой Tor, никакие бридж-базы данных тут вообще не участвуют. Meek-client ожидает подключение типа SOCKS на входе, но игнорирует destination - короче говоря, чтобы превратить это все в простой TCP-туннель, я написал простенький скрипт на Python, который заставляет все это работать как надо:

Скрытый текст

```
`#!/usr/bin/env python3
"""Expose meek-client's ephemeral SOCKS listener as a fixed raw
TCP port."""

import argparse
import os
from pathlib import Path
import signal
import socket
import subprocess
import sys
import tempfile
import threading

def parse_address(value):
    if value.startswith("["):
        host, separator, port = value[1:].partition(":")
    else:
        host, separator, port = value.rpartition(":")
    if not separator or not host or not port:
        raise ValueError(f"invalid address: {value}")
```

```

        return host, int(port)

def recv_exact(sock, length):
    data = bytearray()
    while len(data) < length:
        chunk = sock.recv(length - len(data))
        if not chunk:
            raise ConnectionError("unexpected EOF")
        data.extend(chunk)
    return bytes(data)

def open_meek_tunnel(socks_address):
    sock = socket.create_connection(socks_address)
    try:
        sock.sendall(b"\x05\x01\x00")
        if recv_exact(sock, 2) != b"\x05\x00":
            raise ConnectionError("meek SOCKS listener rejected
no-auth mode")

        # meek-client ignores the requested destination. The real
destination
        # is fixed by TOR_PT_ORPORT on meek-server.
        host = b"ignored.invalid"
        request = b"\x05\x01\x00\x03" + bytes([len(host)]) + host
+ b"\x00\x01"
        sock.sendall(request)
        response = recv_exact(sock, 10)
        if response[:2] != b"\x05\x00":
            raise ConnectionError(f"meek SOCKS CONNECT failed:
{response.hex()}")
        return sock
    except Exception:
        sock.close()
        raise

def pump(source, destination):
    try:
        while True:
            data = source.recv(65536)
            if not data:
                break
            destination.sendall(data)
    except OSError:
        pass
    finally:
        try:
            destination.shutdown(socket.SHUT_WR)
        except OSError:
            pass

def relay(local, remote):
    try:
        threads = [
            threading.Thread(target=pump, args=(local, remote),

```

```

daemon=True),
        threading.Thread(target=pump, args=(remote, local),
daemon=True),
    ]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
finally:
    local.close()
    remote.close()

def start_meek_client(args):
    state = Path(args.state or Path(tempfile.gettempdir()) /
"meek-client-state")
    state = state.resolve()
    state.mkdir(parents=True, exist_ok=True)

    env = os.environ.copy()
    env.update(
        {
            "TOR_PT_MANAGED_TRANSPORT_VER": "1",
            "TOR_PT_STATE_LOCATION": str(state),
            "TOR_PT_CLIENT_TRANSPORTS": "meek",
        }
    )
    for name in (
        "TOR_PT_SERVER_TRANSPORTS",
        "TOR_PT_SERVER_BINDADDR",
        "TOR_PT_SERVER_TRANSPORT_OPTIONS",
        "TOR_PT_ORPORT",
        "TOR_PT_EXTENDED_SERVER_PORT",
        "TOR_PT_AUTH_COOKIE_FILE",
        "TOR_PT_EXIT_ON_STDIN_CLOSE",
    ):
        env.pop(name, None)

    command = [args.meek_client, f"--url={args.url}"]
    if args.front:
        command.append(f"--front={args.front}")
    if args.utls:
        command.append(f"--utls={args.utls}")
    if args.helper:
        command.append(f"--helper={args.helper}")
    if args.proxy:
        command.append(f"--proxy={args.proxy}")
    if args.log:
        command.append(f"--log={args.log}")

    process = subprocess.Popen(
        command,
        env=env,
        stdin=subprocess.DEVNULL,
        stdout=subprocess.PIPE,

```

```

        text=True,
        bufsize=1,
    )

    socks_address = None
    assert process.stdout is not None
    for raw_line in process.stdout:
        line = raw_line.strip()
        print(f"[meek control] {line}", file=sys.stderr)
        fields = line.split()
        if len(fields) == 4 and fields[:3] == ["CMETHOD", "meek",
"socks5"]:
            socks_address = parse_address(fields[3])
            elif line.startswith(("ENV-ERROR ", "VERSION-ERROR ",
"CMETHOD-ERROR ")):
                process.terminate()
                raise RuntimeError(line)
            elif line == "CMETHODS DONE":
                break

    if socks_address is None:
        return_code = process.poll()
        raise RuntimeError(
            f"meek-client did not announce a SOCKS listener;
exit={return_code}"
        )
    return process, socks_address

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--meek-client", required=True)
    parser.add_argument("--url", required=True)
    parser.add_argument("--listen", default="127.0.0.1:7000")
    parser.add_argument("--front")
    parser.add_argument("--utls")
    parser.add_argument("--helper")
    parser.add_argument("--proxy")
    parser.add_argument("--state")
    parser.add_argument("--log")
    parser.add_argument("--once", action="store_true",
help=argparse.SUPPRESS)
    args = parser.parse_args()

    if args.helper and args.utls:
        parser.error("--helper and --utls are mutually
exclusive")

    process, socks_address = start_meek_client(args)
    listen_address = parse_address(args.listen)
    listener = socket.create_server(listen_address)
    stop = threading.Event()

    def request_stop(_signum, _frame):
        stop.set()

```



```

        listener.close()

    signal.signal(signal.SIGINT, request_stop)
    signal.signal(signal.SIGTERM, request_stop)

    print(
        f"raw TCP {listener.getsockname()} -> meek SOCKS
{socks_address}",
        file=sys.stderr,
    )
    try:
        while not stop.is_set():
            try:
                local, _peer = listener.accept()
            except OSError:
                break
            try:
                remote = open_meek_tunnel(socks_address)
            except Exception as error:
                print(f"opening meek tunnel: {error}",
file=sys.stderr)
            local.close()
            continue
            thread = threading.Thread(
                target=relay, args=(local, remote), daemon=not
args.once
            )
            thread.start()
            if args.once:
                thread.join()
                break
        finally:
            listener.close()
            process.terminate()
            try:
                process.wait(timeout=5)
            except subprocess.TimeoutExpired:
                process.kill()
                process.wait()

    if __name__ == "__main__":
        main()

```

Запускать примерно так:

```

python meek_tcp_frontend.py--meek-client meek-
client.exe --url https://reflector.example/meek--utls
HelloFirefox_Auto --listen 127.0.0.1:2222

```

На сервере (тут туннель ведет на локальный SSH):

```
TOR_PT_MANAGED_TRANSPORT_VER=1
```

```
TOR_PT_STATE_LOCATION=/var/lib/meek-state
```

```
TOR_PT_SERVER_TRANSPORTS=meek
```

```
TOR_PT_SERVER_BINDADDR=meek-127.0.0.1:7002
```

```
TOR_PT_ORPORT=127.0.0.1:22
```

```
./meek-server --disable-tls
```

И в итоге подключения на порт 2222 на клиенте будут перенаправлены на порт 22 на сервере.

А если не хочется заморачиваться с этим - meek еще поддерживается в V2ray (от которого в свое время форкнулся всем известный и далеко ушедший от прародителя XRay), но их реализацию я не тестировал.

ntptun

Раз уж в прошлый раз разговор зашел про ICMP и DNS как несущую частоту для туннеля, у них обнаружился еще один малоизвестный родственник - NTP. Встречайте ntptun: IP-over-NTP и UDP-over-NTP туннель на C++.

Идея: полезная нагрузка (целый IP-пакет из TUN-интерфейса, либо UDP-датаграмма) запечатывается в extension field NTP-пакета и улетает как обычный NTP mode-3 запрос. Сервер расшифровывает, достает исходный пакет, отдает его в свой TUN (или на реальный `udp_target`), а ответ прилетает клиенту в теле mode-4 NTP-ответа.

Есть два транспорта. `tun` - классический IP-over-NTP, но только под Linux (нужен `/dev/net/tun`): поднимаете себе `tun0`, вешаете на него внутренний адрес по обе стороны, и через туннель начинает ходить вообще весь IP-трафик, который вы туда завернете, хоть `ping 10.9.0.1` гоняй. И есть UDP-режим (работает под Windows): клиент слушает локальный UDP-порт (`udp_listen`), сервер сливает расшифрованные датаграммы на один реальный `udp_target`, а несколько клиентов на одном UDP-сокетe сервера различаются по исходящему порту `udp_base_port + client_id`.

А теперь самое вкусное - куда это все пристроить. Разработчик рекомендует проксировать протокол KCP поверх NTP используя GOST, так что можно поднять полноценный HTTP/SOCKS5-прокси GOST-а и завернуть его KCP- или DTLS-трафик прямо в `udp`-транспорт `ntptun`:

сервер: `gost -L 'http+kcp://:8443?kcp.mtu=1250'`

клиент: `gost -L http://127.0.0.1:18080 -F 'http+kcp://127.0.0.1:9000?kcp.mtu=1250'`

а между ними уже трудится `ntptun`, который эти KCP/DTLS-пакеты прячет внутрь NTP.

Про маскировку тоже подумали как следует. Downstream (сервер -> клиент) может работать в двух режимах. `poll` (по умолчанию) держит честную семантику NTP запрос/ответ - сервер никогда не шлет ничего незапрошенного, а клиент

постоянно держит в полете пачку пустых поллов (poll_window), чтобы было куда сервером положить ответ, когда тот появится - трафик выглядит один в один как настройка обычного NTP-клиента, но за счет постоянных поллов ест немного CPU и полосы даже в простое. push отбрасывает эту честность - сервер шлет датаграмму сразу, как только она появилась, без запроса, а клиент просто изредка стучится keeralive-ом, чтобы не потерять NAT-маппинг: эффективнее, задержка ниже, но незапрошенные mode-4 пакеты - это то, чего настоящий NTP-сервер никогда не делает, так что придирчивый анализатор трафика такое отличит.

А еще есть защита от active probing (чем-то похожая на XTLS-Reality): если кто-то пошлет на ваш "сервер времени" обычный NTP-запрос (не являющийся валидным туннельным пакетом, либо с неверным ключом), ntptun не промолчит и не ответит подозрительной ошибкой, а перенаправит запрос на реальный upstream NTP-сервер и вернет пробующему настоящий ответ - с рейт-лимитом и защитой от использования как амплификатора.

Собирается стандартно: `cmake -S . -B build && cmake --build build -j`, бинарник - `build/ntptun`, конфиг - обычный текстовый `key = value`. Лицензия GPLv2.